

主流云笔记软件 拆解与自研架构设计

覆盖 10 款产品的开源状态、技术栈与核心功能深度拆解，

基于演进趋势设计新一代云笔记架构

本报告对有道云笔记、思源笔记、Joplin、AppFlowy、Notesnook、印象笔记、为知笔记、EssentialPIM 等 10 款主流云笔记软件进行全方位拆解，聚焦开源状态、技术栈、核心功能与演进潜力，并基于长期可演进性、可扩展性和灵活性设计新一代云笔记架构 Aurora Note。

SCOPE

10 款产品 · 5 大技术栈 · 8 项演进趋势

目录

第一章 执行摘要与核心结论	4
1.1 核心结论速览	4
1.2 报告范围与方法论	4
第二章 主流云笔记软件开源状态总览	6
2.1 开源状态分类总览	6
2.2 技术栈深度拆解	7
2.3 核心功能矩阵	8
2.4 各产品深度点评	9
2.4.1 Joplin — 最成熟的开源 Markdown 笔记	9
2.4.2 AppFlowy — Flutter+Rust 跨端标杆	9
2.4.3 Notesnook — 隐私优先的加密笔记	9
2.4.4 思源笔记 — 国产开源块级笔记	10
2.4.5 Anytype — P2P + CRDT 的革命性架构	10
2.4.6 Obsidian — 插件生态王者	10
2.4.7 有道云笔记 / 印象笔记 — 商业闭源巨头	10
2.4.8 为知笔记 — 支持私有部署的折中方案	10
2.4.9 EssentialPIM — 传统 PIM 的原生之路	11
第三章 五大技术栈深度优劣势分析	12
3.1 技术栈一：Electron + Web (JavaScript/TypeScript)	12
3.2 技术栈二：Flutter + Rust (FFI 桥接)	13
3.3 技术栈三：Tauri 2.0 + Web (Rust 后端)	14
3.4 技术栈四：Qt / C++ (原生跨端)	15
3.5 技术栈五：React Native + 原生 (JavaScript/TypeScript)	16
3.6 五大技术栈综合对比	17

第四章 行业演进趋势分析 19

- 4.1 趋势一：从云中心化到 Local-First 19
- 4.2 趋势二：CRDT 替代 Last-Write-Wins 同步 19
- 4.3 趋势三：块级编辑器成为标配 19
- 4.4 趋势四：WASM 沙箱化插件系统 20
- 4.5 趋势五：AI 能力深度集成 20
- 4.6 趋势六：P2P 同步与去中心化存储 21
- 4.7 趋势七：端到端加密标准化 21
- 4.8 趋势八：跨平台原生体验精细化 21

第五章 Aurora Note 新一代云笔记架构设计 22

- 5.1 设计原则与目标 22
- 5.2 分层架构设计 22
- 5.3 核心数据模型：块级 + CRDT 23
- 5.4 同步架构：混合模式设计 23
- 5.5 端到端加密方案 24
- 5.6 插件系统：WASM 沙箱架构 25
- 5.7 AI 集成架构 25

第六章 技术选型论证与二次开发路线 27

- 6.1 为什么选择 Flutter（移动端）+ Tauri（桌面端）的混合方案 27
- 6.2 为什么选择 Yjs/yrs 作为 CRDT 引擎 27
- 6.3 二次开发路线：基于 AppFlowy 的 Fork 策略 28
- 6.4 技术栈选型完整清单 29

第七章 实施路线图与团队配置 31

- 7.1 四阶段实施路线 31
- 7.2 团队配置建议 31

第八章 风险评估与缓解策略	33
8.1 技术风险	33
8.2 工程风险	33
8.3 生态风险	34
8.4 总结与建议	34

第一章 执行摘要与核心结论

本报告对有道云笔记、思源笔记、Joplin、AppFlowy、Notesnook、印象笔记、为知笔记、EssentialPIM 等 8 款用户指定产品，连同 Obsidian、Anytype 共 10 款主流云笔记软件进行了全方位拆解，聚焦开源状态、技术栈、核心功能与演进潜力四个维度，并结合附件中已有的分析基础，设计了一款名为 Aurora Note 的新一代云笔记架构。在技术栈选型上，报告以"不考虑商用和许可因素，优先考虑长期可演进性、可扩展性、灵活性以及性能和功能"为原则，对五大技术栈进行了深度优劣势分析，并提出在优秀开源软件上二次开发的务实路线。

1.1 核心结论速览

维度	结论
开源格局	10 款产品中 6 款完全开源（Joplin/AppFlowy/Notesnook/思源/Anytype/Obsidian插件生态），3 款闭源（有道/印象/EssentialPIM），为知笔记服务端闭源但支持 Docker 私有部署。开源产品在技术社区活跃度和功能迭代速度上已显著领先。
技术栈分化	五大主流技术栈各具特色：Electron+Web 生态最成熟但资源消耗大；Flutter+Rust 性能与跨端一致性最优；Tauri+Web 轻量但移动端尚在成熟；Qt/C++ 性能极致但开发效率低；React Native+原生 跨端快但性能受限。
同步架构演进	行业正从传统"服务器中心化同步"向"Local-First + CRDT"范式迁移。Anytype 的 any-sync 协议、Automerge/Yjs 等 CRDT 库的成熟，使 P2P + 端到端加密 + 离线优先成为可能。
推荐架构	Aurora Note 采用 Rust 核心 + Tauri 2.0 桌面端 + Flutter 移动端的分层架构，数据层基于 CRDT（Yjs/yrs）实现自动合并，同步层支持自建服务器与 P2P 混合模式，插件系统基于 WASM 沙箱，兼顾安全性与跨平台一致性。
二次开发路线	推荐在 AppFlowy（Flutter+Rust 已验证）基础上进行二次开发，复用其块级编辑器内核和 Rust 数据层，替换同步层为 CRDT 方案，并集成 Tauri 桌面壳，可节省约 40% 的底层开发工作量。

表 1-1 核心结论速览

1.2 报告范围与方法论

本报告的调研覆盖截至 2026 年 7 月的公开信息，包括各产品官方文档、GitHub 仓库代码分析、技术社区讨论及用户反馈。调研维度分为四个层面：第一，开源状态——区分完全开源、部分开源（如仅客户端开源）、闭源但支持私有部署三种状态；第二，技术栈——从前端框架、后端语言、数据存储、同步协议、插件机制五个子维度进行拆解；第三，核心功能——包括编辑

器类型（Markdown/富文本/块级）、同步能力、加密支持、插件生态、多端覆盖等；第四，演进潜力——评估架构的长期可维护性、社区活跃度和功能扩展空间。

在架构设计部分，本报告采用"先分析后设计"的方法论：首先从五大技术栈中提取各自的优劣势和演进趋势，然后以"长期可演进性、可扩展性、灵活性、性能"为四大评估准则，进行综合技术选型。设计过程中参考了附件中已有的三份架构方案（DevNote、Nebula Note、InkFlow 2.0），在吸收其优点的基础上，进一步深化了同步协议设计和插件安全模型。

第二章 主流云笔记软件开源状态总览

本章对 10 款主流云笔记软件的开源状态、技术栈和核心功能进行系统梳理。为便于对比，将产品按开源程度分为三类：完全开源（客户端+服务端均开源）、部分开源（仅客户端开源或服务端闭源但提供私有部署）、完全闭源。这一分类直接影响二次开发的可行性和技术路线选择。

2.1 开源状态分类总览

产品	开源状态	许可证	GitHub Stars	服务端可私有部署
Joplin	完全开源	AGPL-3.0	~46K	是（自建同步服务器）
AppFlowy	完全开源	AGPL-3.0	~58K	是（Supabase/自建）
Notesnook	完全开源	GPL-3.0	~10K	是（自建同步服务器）
思源笔记	完全开源	AGPL-3.0	~18K	是（Docker 私有部署）
Anytype	完全开源	Any Source Available	~4.5K	是（P2P + 自建节点）
Obsidian	闭源（插件生态开放）	—	—	否（仅本地存储）
有道云笔记	闭源	—	—	否
印象笔记	闭源	—	—	否
为知笔记	部分开源（服务端闭源）	—	—	是（Docker 镜像）
EssentialPIM	闭源	—	—	否（仅本地/云盘同步）

表 2-1 10 款云笔记软件开源状态总览

关键发现

在用户关注的 8 款产品中，Joplin、AppFlowy、Notesnook、思源笔记 4 款完全开源，为知笔记支持 Docker 私有部署（部分开源），有道云笔记、印象笔记、EssentialPIM 3 款完全闭源。开源产品在功能迭代速度、社区贡献活跃度和数据自主性方面具有显著优势，而闭源产品在企业级功能（如 OCR、AI 能力）和用户体验打磨上更为成熟。对于自研云笔记而言，4 款完全开源产品是二次开发的优选基础。

2.2 技术栈深度拆解

从技术栈视角看，当前主流云笔记软件可归纳为五大技术栈流派。每一流派在前端框架、后端语言、数据存储和同步机制上各有取舍，直接决定了产品的性能天花板、开发效率和长期演进潜力。以下按技术栈分类进行详细拆解。

产品	前端框架	后端/核心语言	数据存储	同步机制	插件体系
Joplin	React Native (移动) Electron (桌面)	Node.js / TypeScript	SQLite + 文件系统	REST API + WebDAV/ OneDrive/Dropbox	插件系统 (JS)
AppFlowy	Flutter	Rust	SQLite + 文件系统	Supabase / 自建后端	WASM 插件 (规划中)
Notesnook	React Native	JavaScript / TypeScript	SQLite + IndexedDB	自建同步服务器 (E2EE)	无插件体系
思源笔记	Web (TypeScript)	Go	SQLite + JSON 数据块	WebDAV / S3 / 自建	插件系统 (JS)
Anytype	React + Tauri (桌面) React Native (移动)	Go (any-sync)	本地 SQLite + IPFS	P2P + CRDT (any-sync 协议)	无插件体系
Obsidian	Electron + Web	JavaScript / TypeScript	Markdown 文件 + JSON 索引	Obsidian Sync (闭源) 或第三方同步	丰富插件生态 (JS)
有道云笔记	原生 + Web 混合	未知 (推测 Java/C++)	云端存储	自有云同步	无开放插件
印象笔记	原生 + Web 混合	未知 (推测 C++/.NET)	云端存储 + 本地缓存	自有云同步	有限插件支持
为知笔记	Web (Vue.js)	Node.js / Go	MongoDB + 文件系统	自有同步服务器 (Docker 可部署)	有限插件支持
EssentialPIM	Qt / C++ (原生)	C++	本地 SQLite + Firebird	云盘同步 (WebDAV/ Google Drive 等)	无插件体系

表 2-2 10 款云笔记软件技术栈详细拆解

2.3 核心功能矩阵

核心功能为用户选择笔记软件的直接依据，也是自研产品需要对标的功能清单。本节从编辑器类型、多端覆盖、端到端加密（E2EE）、离线支持、AI 功能和协作能力六个维度进行横向对比。其中，编辑器类型决定了内容表达能力——Markdown 适合技术用户但表达力有限，富文本适合大众但难以结构化，块级编辑器（如 Notion/AppFlowy）兼具结构化和灵活性，是当前最先进的编辑器范式。

产品	编辑器类型	桌面端	移动端	E2EE	离线优先	AI 功能	协作
Joplin	Markdown	Win/Mac/Linux	iOS/Android	是	是	插件支持	否
AppFlowy	块级编辑器	Win/Mac/Linux	iOS/Android	规划中	是	内置 AI	是
Notesnook	富文本	Win/Mac/Linux	iOS/Android	是	是	否	否
思源笔记	块级编辑器	Win/Mac/Linux	iOS/Android	是	是	否	有限支持
Anytype	块级编辑器	Win/Mac/Linux	iOS/Android	是	是	否	是（P2P）
Obsidian	Markdown	Win/Mac/Linux	iOS/Android	第三方支持	是	插件支持	第三方支持
有道云笔记	富文本	Win/Mac	iOS/Android	否	部分	内置 AI	是
印象笔记	富文本	Win/Mac	iOS/Android	否	部分	内置 AI	是
为知笔记	Markdown/富文本	Win/Mac/Linux	iOS/Android	否	部分	否	是
Essential PIM	富文本	Win	Android	否	是	否	否

表 2-3 核心功能矩阵

2.4 各产品深度点评

2.4.1 Joplin — 最成熟的开源 Markdown 笔记

Joplin 是开源笔记软件中历史最悠久、社区最活跃的产品之一。它采用 AGPL-3.0 许可证，客户端完全开源，同步服务器可以自建或使用 WebDAV/OneDrive/Dropbox 等第三方云存储。技术栈上，桌面端基于 Electron，移动端基于 React Native，后端逻辑使用 Node.js/TypeScript 编写，数据存储在本地 SQLite 数据库中。Joplin 的最大优势在于成熟稳定——经过多年迭代，其 Markdown 编辑、标签管理、全文搜索、端到端加密等核心功能已非常完善，插件生态虽不及 Obsidian 丰富但也有数百个社区插件。

Joplin 的主要局限在于技术栈较为传统。Electron 带来了较高的内存占用，React Native 在 iOS/Android 上的编辑体验与原生应用有差距。更重要的是，Joplin 的同步模型仍基于传统的“最后写入胜出”（Last-Write-Wins）策略，在多设备同时编辑时可能产生冲突。它没有采用 CRDT 等自动合并技术，这在协作场景下是明显的短板。对于自研项目而言，Joplin 的代码结构和功能设计值得参考，但其技术栈本身并非最先进的选择。

2.4.2 AppFlowy — Flutter+Rust 跨端标杆

AppFlowy 是一款定位为“Notion 开源替代品”的产品，采用 Flutter 前端 + Rust 后端的技术栈，在 GitHub 上获得了超过 58K Star。其核心创新在于将 UI 层完全交给 Flutter（实现真正的跨端一致性：一套代码覆盖桌面+移动+Web），将数据层和业务逻辑完全交给 Rust（实现高性能、内存安全和线程安全）。两者通过 FFI（Foreign Function Interface）桥接。AppFlowy 使用块级编辑器（Block-based Editor），支持数据库视图（Grid/Kanban/Calendar），功能上对标 Notion。

AppFlowy 的技术选型非常前瞻：Rust 核心保证了数据处理的安全性和性能，Flutter 保证了 UI 的跨端一致性。其编辑器架构基于自定义的块级数据模型，每个块（Block）是一个独立的数据单元，支持嵌套和引用。同步方面，AppFlowy 目前依赖 Supabase 作为后端，但也支持完全自建。AppFlowy 的不足之处在于：端到端加密尚未实现，同步冲突解决策略仍基于简单的版本号比较，插件系统尚未成熟。尽管如此，AppFlowy 的 Flutter+Rust 架构已经过大规模验证，是自研云笔记最值得借鉴的二次开发基础。

2.4.3 Notesnook — 隐私优先的加密笔记

Notesnook 是一款以隐私保护为核心卖点的开源笔记应用，采用 GPL-3.0 许可证。它的最大特色是全面的端到端加密（E2EE）——所有笔记内容在客户端加密后才上传到服务器，即使服务器被攻破也无法读取用户数据。技术栈上，Notesnook 使用 React Native 构建跨平台客户端，后端使用 Node.js，数据存储在本地 SQLite 和 IndexedDB 中。同步服务器可以自建，完全开源。

Notesnook 的加密实现非常完善，采用 XChaCha20-Poly1305 对称加密和 X25519 密钥交换，支持密钥分离和可验证加密。但在编辑器方面，Notesnook 使用传统的富文本编辑器而非块级编辑器，内容表达力不如 AppFlowy 或 Notion。同时 React Native 在桌面端的体验有

限（需借助 React Native for Web 或 Electron），性能不如原生方案。Notesnook 的加密方案设计值得自研项目深度参考，但其整体技术栈并非最优选择。

2.4.4 思源笔记 — 国产开源块级笔记

思源笔记是一款国产开源笔记软件，采用 AGPL-3.0 许可证，在 GitHub 上有约 18K Star。技术上，前端使用 TypeScript + Web 技术，后端使用 Go 语言，数据存储 in SQLite 中，以 JSON 格式组织数据块。思源笔记的核心特色是“块级引用”——每个内容块都有唯一 ID，可以在笔记间双向链接和引用，形成知识图谱。它支持 Docker 私有部署，同步方式灵活（WebDAV/S3/自建服务器），也支持端到端加密。

思源笔记的 Go 后端在性能上表现优异，JSON 数据块模型灵活可扩展。其不足在于前端完全基于 Web 技术，桌面端通过 Electron 或浏览器运行，移动端通过 WebView 包装，性能和原生体验不如 Flutter 方案。此外，思源的同步机制仍基于传统的“全量同步+冲突检测”，未采用 CRDT 自动合并。对于自研项目，思源的块级数据模型设计（JSON 格式、ID 管理、双向链接）和 Go 后端的工程实践值得参考。

2.4.5 Anytype — P2P + CRDT 的革命性架构

Anytype 采用了与所有其他产品都不同的架构路线：基于自研的 any-sync 协议实现 P2P（点对点）同步，不依赖中心化服务器。数据存储在本地，通过 CRDT（Conflict-free Replicated Data Types）实现多设备自动合并，端到端加密由协议层保证。前端使用 React + Tauri（桌面）和 React Native（移动），后端协议使用 Go 实现。Anytype 的架构是当前最先进的 Local-First 笔记架构，但其复杂度也很高，且社区规模相对较小。

2.4.6 Obsidian — 插件生态王者

Obsidian 本身是闭源软件，但其插件生态是所有笔记软件中最丰富的——超过 2000 个社区插件覆盖了从学术写作到项目管理的各种场景。技术上，Obsidian 基于 Electron + Web 技术，数据以 Markdown 文件形式存储在本地，完全离线优先。其同步服务 Obsidian Sync 是闭源付费服务，但用户也可以使用第三方同步方案（如 Git、Syncthing 等）。Obsidian 的插件架构设计（基于 JavaScript API 暴露内部能力）值得自研项目参考。

2.4.7 有道云笔记 / 印象笔记 — 商业闭源巨头

有道云笔记和印象笔记都是国内市场的老牌笔记产品，完全闭源，采用自有云同步。两者的技术栈虽未公开，但从产品行为推测，客户端采用原生开发 + Web 混合方案，后端采用 Java/C++ 等传统企业技术栈。它们的优势在于成熟的企业级功能（OCR 文字识别、文档扫描、AI 辅助写作等）和大规模用户基础，但在数据自主性、跨平台开放性和隐私保护方面存在明显短板。对于自研项目，这两款产品的功能设计（尤其是 AI 集成和移动端体验）可作为产品层面的参考，但技术架构参考价值有限。

2.4.8 为知笔记 — 支持私有部署的折中方案

为知笔记（WizNote）采用部分开源策略：客户端代码部分开源，服务端闭源但提供 Docker 镜像支持私有部署。技术上，前端使用 Vue.js，后端使用 Node.js/Go，数据存储在 MongoDB 中。为知笔记是国内最早支持私有部署的笔记产品，在企业知识管理场景有一定用户基础。但其技术栈较为传统（Vue.js + MongoDB），编辑器仍以 Markdown 和富文本为主，未采用块级编辑器和 CRDT 等先进技术。私有部署的 Docker 方案设计值得参考。

2.4.9 EssentialPIM — 传统 PIM 的原生之路

EssentialPIM 是一款老牌个人信息管理（PIM）软件，采用 Qt/C++ 原生开发，仅支持 Windows 桌面和 Android 移动端。它的特色是将笔记、日历、联系人、任务等多种个人信息整合在一个应用中，数据存储在本地 SQLite/Firebird 数据库，通过 WebDAV 或云盘进行同步。EssentialPIM 的 Qt/C++ 技术栈带来了极致的运行性能和低资源占用，但开发效率低、跨端能力弱（仅 Win+Android）、无插件生态，且缺乏现代笔记功能（块级编辑、CRDT 同步、AI 集成等）。其原生性能优化和离线优先理念值得参考，但整体技术路线已不适合新一代云笔记的开发。

第三章 五大技术栈深度优劣势分析

本章从云笔记应用的特殊需求出发，对五大主流技术栈进行深度优劣势分析。云笔记应用的技术挑战具有独特性：它既需要富文本/块级编辑器的高频 UI 交互（要求前端框架灵活高效），又需要本地数据持久化和全文搜索（要求核心层高性能），还需要多设备同步和冲突合并（要求同步协议精密可靠），同时要覆盖桌面和移动端（要求跨端一致性）。这些需求使得技术栈选型不能简单看"流行度"，而需要从多个维度进行系统评估。

3.1 技术栈一：Electron + Web（JavaScript/TypeScript）

代表产品：Obsidian、Joplin（桌面端）、印象笔记（部分）。这是目前最成熟的桌面笔记应用技术栈，核心思路是使用 Chromium 渲染引擎 + Node.js 运行时打包 Web 应用为桌面程序。前端使用 HTML/CSS/JavaScript（或 TypeScript），可借助 React/Vue/Svelte 等 Web 框架。编辑器通常基于 ProseMirror、CodeMirror 或 Lexical 等 Web 编辑器框架。

优势	详细说明
生态最成熟	Web 生态拥有最丰富的开源库和工具链。ProseMirror/CodeMirror/Lexical 等编辑器框架经过大规模生产验证，文档和社区支持完善。React/Vue 等框架的开发效率极高，前端人才储备充足。
插件开发门槛低	基于 JavaScript 的插件系统让社区贡献者可以快速开发插件。Obsidian 超过 2000 个插件正是得益于此。插件可以使用 npm 生态的全部包，开发体验接近普通 Web 开发。
UI 一致性高	Electron 在 Win/Mac/Linux 上使用相同的 Chromium 引擎，UI 渲染一致性极高，无需处理各平台原生 UI 差异。CSS 的灵活布局能力让复杂编辑器界面的实现比原生方案高效得多。

表 3-1 Electron+Web 技术栈优势

劣势	详细说明
资源消耗大	每个 Electron 应用都捆绑了完整的 Chromium + Node.js，内存占用通常在 200-500MB 以上，安装包体积 80-150MB。这对于移动端是不可接受的——这也是为什么 Electron 技术栈无法扩展到移动端的原因。
移动端空白	Electron 仅适用于桌面端。移动端需要使用 React Native 或其他方案重新开发，导致前后端代码分离、编辑器逻辑需要两套实现、同步逻辑需要跨语言桥接。Joplin 正是这种"桌面 Electron + 移动 React Native"的双栈架构，维护成本高。

劣势	详细说明
性能天花板	JavaScript 是动态类型语言，V8 引擎虽然有 JIT 优化，但在数据处理密集型场景（如全文搜索索引构建、大文档渲染、加密计算）下性能仍显著低于 Rust/C++。对于万条笔记以上的库，搜索和索引性能会明显下降。
安全面广	Electron 应用暴露了完整的 Node.js API 和 Chromium 渲染进程，攻击面较广。历史上多次出现通过恶意笔记内容触发 XSS 进而获取文件系统访问权限的安全漏洞。插件运行在与主进程相同的上下文中，一个恶意插件可以访问全部用户数据。

表 3-2 Electron+Web 技术栈劣势

3.2 技术栈二：Flutter + Rust（FFI 桥接）

代表产品：AppFlowy。这是当前最具前瞻性的跨端笔记技术栈，核心思路是将 UI 层交给 Flutter（一套代码覆盖 iOS/Android/Windows/macOS/Linux/Web），将数据层和业务逻辑交给 Rust（高性能、内存安全、线程安全），两者通过 FFI（Foreign Function Interface）或 flutter_rust_bridge 桥接。这种分层使得 UI 和数据完全解耦——Flutter 只负责渲染和交互，Rust 只负责数据处理和存储，各自发挥所长。

优势	详细说明
真正的跨端一致性	Flutter 的 Skia/Impeller 渲染引擎在所有平台上自绘 UI，不依赖原生控件，因此实现了像素级一致的渲染效果。一套 Dart 代码可以同时编译为 iOS/Android/Win/Mac/Linux 应用，甚至通过 Flutter Web 编译为 Web 应用。这是目前跨端覆盖最全面的方案。
Rust 核心高性能	Rust 的性能接近 C/C++，在数据处理、加密计算、全文索引等密集型场景下远超 JavaScript。同时 Rust 的所有权系统在编译期消除了空指针、数据竞争等常见内存安全问题，适合处理用户敏感数据。
单一代码库	与 Electron+React Native 方案相比，Flutter+Rust 方案的桌面端和移动端共享同一套 Flutter UI 代码和同一套 Rust 核心代码。编辑器逻辑只需要实现一次，在各端的表现完全一致。这大幅降低了维护成本。
包体积小	Flutter 应用的安装包通常在 20-40MB（移动端）和 40-60MB（桌面端），远小于 Electron 的 80-150MB。Rust 核心编译为原生机器码，无运行时依赖。

表 3-3 Flutter+Rust 技术栈优势

劣势	详细说明
桌面端成熟度	Flutter 的移动端已经非常成熟（大量生产应用验证），但桌面端虽然已发布稳定版，在复杂文本编辑器场景下仍有边缘 case 需要处理。例如：复杂的文本选择、拖拽、快捷键处理在桌面端的行为与移动端不同，需要额外适配。

劣势	详细说明
编辑器生态薄弱	Flutter 生态中尚无成熟的块级编辑器框架（对标 Web 端的 ProseMirror）。AppFlowy 自研了整个块级编辑器，投入了大量工程资源。这意味着选择 Flutter+Rust 的团队需要准备自研编辑器或基于 AppFlowy 编辑器二次开发。
FFI 复杂性	Flutter 与 Rust 通过 FFI 桥接（通常使用 flutter_rust_bridge 工具），虽然工具链已经比较成熟，但在复杂数据结构传递、异步回调、错误处理等方面仍有额外开发成本。调试 FFI 边界问题比纯 JavaScript 或纯 Dart 更困难。
人才稀缺	同时精通 Flutter 和 Rust 的工程师较少。Dart 语言虽然易学，但 Rust 的学习曲线陡峭（所有权、生命周期、借用检查器等概念需要时间消化）。团队组建和人才培养成本较高。

表 3-4 Flutter+Rust 技术栈劣势

3.3 技术栈三：Tauri 2.0 + Web（Rust 后端）

代表产品：Anytype（桌面端部分使用）。Tauri 是一个新兴的跨端框架，理念类似 Electron 但使用系统原生 WebView 替代捆绑 Chromium，后端使用 Rust 替代 Node.js。Tauri 2.0（2024 年 10 月发布稳定版）新增了移动端支持（iOS/Android），使其成为 Electron 的轻量替代方案。前端仍使用 Web 技术（HTML/CSS/JS + React/Vue/Svelte 等），后端用 Rust 编写，通过 Tauri Commands 桥接。

优势	详细说明
极致轻量	Tauri 应用使用系统原生 WebView（macOS 用 WebKit、Windows 用 WebView2、Linux 用 WebKitGTK），不捆绑浏览器引擎，安装包通常仅 3-10MB，内存占用 50-100MB，相比 Electron 减少了一个数量级。
Rust 安全性	后端使用 Rust，天然抵御内存安全漏洞。Tauri 的权限系统比 Electron 更精细——可以精确控制每个 Tauri Command 的访问权限，插件/前端代码只能调用被授权的接口，安全性大幅提升。
2.0 移动端支持	Tauri 2.0 同时支持桌面端和移动端，前端代码可以共享。虽然移动端仍处于早期阶段，但官方投入了大量资源，社区增长迅速。对于笔记应用，Tauri 2.0 提供了"一套 Web 前端 + Rust 后端覆盖全平台"的可能性。
Web 前端生态	前端使用 Web 技术，可以直接复用 ProseMirror/Lexical 等成熟编辑器框架和 React/Vue 生态的全部组件库，编辑器开发效率远高于 Flutter 方案。

表 3-5 Tauri+Web 技术栈优势

劣势	详细说明
WebView 碎片化	不同平台使用不同 WebView 引擎（WebKit/WebView2/WebKitGTK），存在 CSS 渲染差异和 API 兼容性问题。Linux 上的 WebKitGTK 版本碎片化尤其严重，需要针对不同发行版测试。这是 Tauri 最持久的痛点。
移动端不成熟	Tauri 2.0 的移动端支持虽然已发布稳定版，但相比 Flutter 的移动端成熟度仍有明显差距。复杂手势、列表滚动、文本编辑等移动端核心交互在 Tauri 中的体验尚不如 Flutter 或 React Native。
性能折中	Web 前端本身有 JavaScript 性能开销，虽然后端用 Rust 可以处理密集计算，但 UI 层的渲染性能仍受限于 WebView。在复杂编辑器场景（大量 DOM 节点、频繁重绘）下，性能可能不如 Flutter 的自绘引擎。
生态规模小	Tauri 的插件和工具生态虽然增长很快，但与 Electron 的成熟生态相比仍有数量级差距。很多 Electron 开箱即用的功能在 Tauri 中需要自己实现。

表 3-6 Tauri+Web 技术栈劣势

3.4 技术栈四：Qt / C++（原生跨端）

代表产品：EssentialPIM。Qt 是历史悠久的跨平台 C++ GUI 框架，通过一次编写可以编译为 Win/Mac/Linux 甚至移动端应用。Qt 提供了完整的控件库、布局系统、数据模型和信号槽机制，配合 C++ 的高性能，适合构建资源敏感型桌面应用。近年来 Qt Quick（QML 声明式 UI）的加入使得 UI 开发效率有所提升。

优势	详细说明
性能极致	C++ 编译为原生机器码，无运行时开销，在所有技术栈中性能最优、内存占用最低。EssentialPIM 的内存占用仅 30-50MB，启动时间小于 1 秒，远优于 Electron/Tauri 方案。
原生体验	Qt 调用各平台原生控件和 API，可以提供最接近原生的用户体验。文件系统集成、系统通知、全局快捷键等桌面特性的实现最为自然。

表 3-7 Qt/C++ 技术栈优势

劣势	详细说明
开发效率低	C++ 是手动内存管理语言，开发效率远低于 JavaScript/Dart/Rust。缺乏现代语言的 ergonomics（如模式匹配、闭包、async/await 的简洁语法），代码量通常是同等功能 JavaScript 的 2-3 倍。

劣势	详细说明
移动端弱	Qt 虽然支持移动端，但在移动端的实际体验远不如 Flutter/React Native。Qt 的移动端应用在 UI 流畅度、手势处理、平台集成等方面都有明显短板，市场上基于 Qt 的成功移动应用寥寥无几。
人才稀缺且昂贵	精通 Qt 的 C++ 工程师在市场上非常稀缺，薪资水平高。团队扩展困难，代码维护门槛高（C++ 代码的可读性和可维护性很大程度上依赖开发者水平）。
编辑器生态空白	Qt 生态中没有任何成熟的富文本/块级编辑器组件。自研编辑器需要从零开始实现文本排版、光标管理、选择高亮等所有基础功能，工作量巨大。
插件系统困难	C++ 的 ABI 不稳定，动态加载插件非常困难（跨编译器、跨版本的 ABI 兼容性问题）。这使得基于 Qt 的笔记应用几乎不可能实现像 Obsidian 那样丰富的插件生态。

表 3-8 Qt/C++ 技术栈劣势

3.5 技术栈五：React Native + 原生（JavaScript/TypeScript）

代表产品：Joplin（移动端）、Notesnook。React Native 是 Facebook 开发的跨平台移动框架，使用 JavaScript 编写 UI 和业务逻辑，编译为原生组件渲染。通过 React Native for Web 或 Electron 可以扩展到桌面端。核心思路是"Learn once, write anywhere"——一套 JavaScript 代码在不同平台上渲染为各自的原生组件。

优势	详细说明
移动端成熟	React Native 是经过大规模验证的移动端跨平台方案，大量知名应用（Instagram/Discord/Shopify 等）使用它。移动端的 UI 流畅度、手势处理、平台集成（相机/通知/分享等）都很成熟。
Web 生态复用	React Native 使用 JavaScript/TypeScript，可以直接复用 npm 生态的大量库。开发者可以从 Web 前端平滑过渡到移动开发，人才储备充足。
热更新能力	JavaScript 代码可以动态下发（在允许的平台和合规框架下），支持热更新，不需要重新发布应用商店版本。这对于快速修复 bug 和迭代功能非常有价值。

表 3-9 React Native 技术栈优势

劣势	详细说明
桌面端鸡肋	React Native 在桌面端的体验远不如移动端。虽然可以通过 react-native-windows 扩展到 Windows，或通过 Electron 包装扩展到 Mac/Linux，但前者生态薄弱、后者退化为 Electron 方案。桌面端和移动端使用不同的技术栈是 React Native 方案的常见痛点。

劣势	详细说明
JavaScript Bridge 开销	React Native 通过 Bridge 在 JavaScript 线程和原生线程之间通信。虽然新架构（Fabric/TurboModules）减少了开销，但在高频交互场景（如复杂编辑器）下，Bridge 通信仍是性能瓶颈。
编辑器自研成本	React Native 生态中没有成熟的块级编辑器框架。Notesnook 的富文本编辑器是基于 react-native-pell-rich-editor 等基础组件自研的，功能有限。要实现 Notion 级别的块级编辑器，投入巨大。
原生模块维护	很多功能（如文件系统、加密、后台同步）需要编写原生模块（Java/Kotlin for Android, Swift/ObjC for iOS）。这意味着团队仍需具备各平台原生开发能力，跨端一致性难以保证。

表 3-10 React Native 技术栈劣势

3.6 五大技术栈综合对比

基于以上逐项分析，下表从七个关键维度对五大技术栈进行综合评分（5 分制），为后续架构选型提供量化依据。评分标准：5=卓越，4=良好，3=中等，2=不足，1=严重不足。

维度	Electron+Web	Flutter+Rust	Tauri+Web	Qt/C++	RN+原生
跨端一致性	3 (桌面强/移动弱)	5 (全端一致)	3 (桌面强/移动新)	2 (桌面强/移动弱)	3 (移动强/桌面弱)
性能	2	5	3	5	3
包体积/资源	1	4	5	5	3
编辑器生态	5 (ProseMirror等)	2 (需自研)	5 (Web生态)	1 (空白)	2 (基础组件)
开发效率	5	3	4	2	4
插件安全	2	4 (WASM规划)	4 (权限系统)	1 (ABI困难)	2
长期演进	3 (移动端缺失)	5 (全栈现代)	4 (快速成长)	2 (技术老化)	3 (桌面受限)
综合评分	22/35	28/35	28/35	18/35	20/35

表 3-11 五大技术栈综合评分对比（满分 35 分）

技术栈选型结论

Flutter+Rust 和 Tauri+Web 以并列最高分（28/35）成为最优技术栈选择。Flutter+Rust 在跨端一致性和性能上更胜一筹，适合需要全端覆盖且追求极致体验的场景；Tauri+Web 在轻量性和编辑器生态复用上更有优势，适合以 Web 技术栈为主的团队。Aurora Note 架构采用混合方案：移动端使用 Flutter（成熟的移动体验），桌面端使用 Tauri 2.0（轻量+Web 编辑器生态），核心数据层使用 Rust 统一实现，通过 FFI 桥接两端，兼顾各自优势。

第四章 行业演进趋势分析

技术栈选型不仅要看当下的优劣，更要判断未来的演进方向。本章从架构范式、同步协议、编辑器模型、插件安全和 AI 集成五个维度，分析云笔记行业的演进趋势，为 Aurora Note 的架构设计提供前瞻性指引。

4.1 趋势一：从云中心化到 Local-First

传统云笔记（如有道云笔记、印象笔记）采用"云中心化"架构——数据存储在云端，客户端只是缓存层，所有操作需要网络连接。这种架构的痛点是：离线时功能严重受限、网络延迟影响编辑体验、用户数据掌握在服务商手中。新一代笔记应用（Obsidian/Anytype/AppFlowy/思源）已全面转向 Local-First 架构——数据优先存储在本地，应用可以完全离线工作，同步是后台异步行为而非前置依赖。这一趋势的核心推动力是移动设备本地存储能力的提升和用户对数据自主权意识的增强。

Local-First 不等于"不同步"，而是将同步从"核心路径"降级为"增强路径"。用户的所有编辑操作直接写入本地数据库，UI 响应不受网络状态影响。同步在后台进行，即使同步失败也不影响本地使用。这种架构对数据层设计有深远影响：需要支持离线写入+延迟同步的数据模型、需要处理多设备并发编辑的冲突合并、需要保证本地数据完整性（即使同步过程中断也不损坏数据）。CRDT（无冲突复制数据类型）正是为这一场景而生的技术，它能在网络分区的情况下自动合并多端修改，不产生冲突。

4.2 趋势二：CRDT 替代 Last-Write-Wins 同步

传统的笔记同步采用"最后写入胜出"（Last-Write-Wins, LWW）策略——每条笔记有版本号或时间戳，同步时取最新版本覆盖旧版本。这种策略在单设备编辑时工作良好，但在多设备同时编辑同一条笔记时会导致数据丢失——先保存的设备的修改会被后保存的设备覆盖。CRDT（Conflict-free Replicated Data Types）通过数学上保证的数据结构设计，使得任何顺序的修改操作都能自动合并且不产生冲突，是解决多端同步问题的终极方案。

当前主流的 CRDT 实现包括：Yjs（JavaScript，最流行）及其 Rust 移植 yrs、Automerge（JavaScript + Rust 核心）、Diamond Types（Swift）等。Anytype 的 any-sync 协议在底层使用了自定义的 CRDT 实现，支持文本、富文本、结构化数据等多种数据类型的自动合并。AppFlowy 也在规划中引入 CRDT 替换当前的简单同步策略。预计未来 2-3 年内，CRDT 将成为云笔记同步的标准方案，就像 Git 之于代码版本管理一样。

4.3 趋势三：块级编辑器成为标配

编辑器是笔记软件的核心，其架构直接决定了内容表达能力。传统笔记编辑器分为两类：Markdown 编辑器（如 Joplin/Obsidian）和富文本编辑器（如有道云笔记/印象笔记）。Markdown 编辑器对技术用户友好但表达力有限（难以嵌入表格、数据库等复杂结构），富文本编辑器对大众友好但难以结构化和迁移。Notion 开创的"块级编辑器"（Block-based Editor）范式将内容组织为独立的块（Block），每个块可以是段落、标题、图片、代码、表格、数据库等任意类型，块之间可以拖拽排序、嵌套、引用，兼具结构化和灵活性。

块级编辑器的核心技术挑战是：块的数据模型设计（如何表示嵌套和引用）、块的渲染性能（大量块时的虚拟滚动）、块的协同编辑（多人同时编辑同一块时的冲突合并）、块的序列化（如何在同步和存储中保持块结构完整性）。当前成熟的块级编辑器实现包括：ProseMirror（Web, Obsidian/Notion 等使用其理念）、Lexical（Web, Meta 开源）、AppFlowy Editor（Flutter, AppFlowy 自研）。自研云笔记应直接采用或基于这些成熟方案二次开发，而非从零构建。

4.4 趋势四：WASM 沙箱化插件系统

插件系统是笔记软件功能扩展的关键机制，但传统插件系统面临两难：如果给予插件完整 API 访问权限（如 Obsidian），安全风险高——恶意插件可以访问全部用户数据；如果严格限制插件权限（如移动端应用沙箱），则功能受限，复杂插件无法实现。WASM（WebAssembly）沙箱提供了一条折中路线：插件编译为 WASM 字节码，在沙箱中运行，通过能力声明（Capability-based Security）精确控制每个插件可以访问的资源。

WASM 沙箱的优势包括：语言无关——插件可以用任何能编译为 WASM 的语言编写（Rust/C/C++/Go/AssemblyScript 等）；安全隔离——WASM 运行在独立内存空间，无法直接访问宿主内存；性能接近原生——WASM 字节码接近机器码效率，远优于 JavaScript 插件；能力声明——通过 wasmtime 等运行时的能力系统，可以精确控制每个插件的文件系统、网络、数据库访问权限。AppFlowy 已在规划 WASM 插件系统，预计将成为新一代笔记应用的插件标准方案。

4.5 趋势五：AI 能力深度集成

AI 已从笔记软件的"附加功能"演进为"核心能力"。有道云笔记和印象笔记已内置 AI 写作辅助、AI 摘要、AI 问答等功能。Obsidian 通过插件（如 Smart Connections）实现 AI 问答和知识图谱关联。未来笔记应用的 AI 集成将更加深度：本地 RAG（Retrieval-Augmented Generation）——在用户笔记库中构建向量索引，AI 问答可以直接引用用户自己的笔记内容；AI 辅助编辑——在写作过程中提供实时的内容补全、改写建议、事实核查；智能组织——AI 自动为笔记打标签、生成大纲、推荐关联笔记。

AI 集成对架构设计的影响包括：需要在本地维护向量索引（如使用 SQLite-VSS 或 LanceDB 等嵌入式向量数据库），支持本地推理（通过 ONNX Runtime 或 llamacpp 运行量

化小模型）或云端推理（调用 OpenAI/Claude 等 API）；需要设计 AI 上下文管理机制——如何从用户笔记库中检索相关内容作为 AI 上下文，如何保护隐私（敏感内容不被发送到云端 API）。Aurora Note 将在架构层面预留 AI 集成接口，支持本地和云端混合推理模式。

4.6 趋势六：P2P 同步与去中心化存储

Anytype 的 any-sync 协议开创了 P2P（点对点）同步在笔记应用中的实践——设备之间直接同步数据，不依赖中心化服务器。P2P 同步的优势是：无服务器成本、无单点故障、天然抗审查、用户完全掌握数据。其挑战是：NAT 穿透需要协调服务器（STUN/TURN）、设备发现需要引导节点、离线设备的数据需要中继存储。any-sync 协议通过“协调节点 + 存储节点 + 文件节点”的分层设计解决了这些问题，同时保持端到端加密。

P2P 同步不会完全替代 C/S（客户端/服务器）同步，但会作为一种重要选项。理想的同步架构应支持混合模式：日常同步走自建服务器（稳定可靠），特殊场景下可以启用 P2P 直连（如两台设备在同一局域网内大文件同步）。IPFS（InterPlanetary File System）作为去中心化存储方案，可以用于存储笔记中的大文件（如图片、附件），通过内容寻址（Content Addressing）保证数据完整性。Anytype 已在底层使用 IPFS 进行文件存储。

4.7 趋势七：端到端加密标准化

端到端加密（E2EE）已从“高级功能”变为“标配能力”。Notesnook、Anytype、思源笔记等开源产品均已实现完善的 E2EE，而有道云笔记和印象笔记等闭源产品仍未提供。E2EE 的技术挑战不在加密本身（XChaCha20-Poly1305 等算法已标准化），而在于密钥管理：如何在多设备间安全共享密钥、如何处理设备丢失/更换、如何支持密码恢复。Notesnook 的密钥管理方案（基于用户密码的密钥派生 + 设备密钥 + 密钥分享协议）是当前最完善的设计，值得参考。

4.8 趋势八：跨平台原生体验精细化

早期跨平台笔记应用往往在各平台上体验参差不齐——桌面端功能丰富但移动端简陋，或移动端流畅但桌面端粗糙。新一代用户期望在各平台获得“原生级”体验：移动端需要流畅的手势操作、离线支持、后台同步、推送通知；桌面端需要快捷键体系、多窗口/分屏、系统级集成（托盘/全局快捷键/右键菜单）。这要求架构设计在“共享核心逻辑”和“平台原生适配”之间找到平衡——核心编辑器和数据层跨端共享，但 UI 交互层针对各平台做精细化适配，而非简单地一套 UI 跑所有平台。

第五章 Aurora Note 新一代云笔记架构设计

基于前四章的分析——10 款产品的开源状态与技术栈拆解、五大技术栈的优劣势对比、八大行业演进趋势——本章设计一款名为 Aurora Note 的新一代云笔记架构。设计原则是：不考虑商用和许可因素，优先考虑长期可演进性、可扩展性、灵活性以及性能和功能，支持桌面端和移动端（Web 端现阶段非必须），可在优秀开源软件上二次开发。

5.1 设计原则与目标

原则	具体目标
长期可演进性	架构分层清晰，各层可独立演进替换。核心数据层使用 Rust 实现，保证 5-10 年代码可维护性。数据格式基于开放的 JSON Schema，不绑定特定数据库引擎。
可扩展性	插件系统基于 WASM 沙箱，支持第三方扩展。数据模型支持自定义块类型，允许插件注册新的内容块。AI 接口预留，支持接入不同的本地/云端 AI 模型。
灵活性	同步层支持多种后端：自建服务器、P2P 直连、第三方云存储（WebDAV/S3）。加密可选（用户可选择是否启用 E2EE）。编辑器支持 Markdown 和块级两种模式。
性能	本地操作零延迟（直接写入 SQLite）。全文搜索基于 Rust 实现的倒排索引，万条笔记搜索 <50ms。大文档（10万字符）编辑帧率 >55fps。冷启动 <2 秒。
功能完整性	对标 Notion + Obsidian 的功能集：块级编辑器、双向链接、数据库视图、全文搜索、E2EE、多端同步、插件系统、AI 辅助。
全端覆盖	桌面端覆盖 Win/Mac/Linux，移动端覆盖 iOS/Android。Web 端为可选项（预留架构接口但不在首期实现）。

表 5-1 Aurora Note 设计原则与目标

5.2 分层架构设计

Aurora Note 采用五层架构，自底向上依次为：数据持久层、核心逻辑层、同步与加密层、UI 桥接层、平台 UI 层。每一层都有明确的职责边界和接口定义，下层不依赖上层，同层模块通过定义良好的接口通信。这种分层确保了各层可以独立演进——例如可以替换 UI 层从 Flutter 切换到 Tauri 而不影响核心逻辑层，或替换同步层从 C/S 模式切换到 P2P 而不影响数据持久层。

层级	职责	技术选型	关键模块
L1 数据持久层	本地数据存储、索引、全文搜索	Rust + SQLite + SQLite-VSS(向量索引)	schema.rs (表结构) fts.rs (全文搜索) vector.rs (向量索引)
L2 核心逻辑层	笔记CRUD、块级模型、编辑器逻辑	Rust	block.rs (块模型) editor.rs (编辑器引擎) parser.rs (Markdown解析)
L3 同步与加密层	多端同步、CRDT 合并、E2EE	Rust + yrs(Yjs CRDT) + XChaCha20-Poly1305	sync.rs (同步引擎) crdt.rs (冲突合并) crypto.rs (加密模块)
L4 UI 桥接层	核心层与UI层的FFI桥接	flutter_rust_bridge + Tauri Commands	bridge.rs (Flutter桥接) tauri_cmds.rs (Tauri命令)
L5 平台UI层	用户界面渲染与交互	移动端: Flutter 桌面端: Tauri+React	editor_widget.dart desktop_app.tsx

表 5-2 Aurora Note 五层架构设计

5.3 核心数据模型：块级 + CRDT

Aurora Note 的核心数据模型采用"块级 + CRDT"设计——每条笔记由若干块（Block）组成，每个块是一个独立的 CRDT 文档，可以独立同步和合并。块的基本结构如下：

```
Block {  
  id: UUID, // 全局唯一标识  
  type: BlockType, // paragraph/heading/code/table/database/...  
  content: CRDTDoc, // Yjs/Yrs CRDT 文档，支持自动合并  
  children: Vec<BlockID>, // 嵌套子块  
  props: JSON, // 块级属性（对齐/颜色/折叠状态等）  
  refs: Vec<BlockID>, // 双向链接  
  created_at: Timestamp,  
  updated_at: Timestamp,  
}
```

每个块的内容（content 字段）是一个独立的 CRDT 文档，使用 Yjs/yrs 库实现。这意味着当两个设备同时编辑同一个块的不同部分时，修改可以自动合并不产生冲突。块级别的操作（增删块、排序块、移动块到不同父块）也通过 CRDT 序列化，确保多端同步时块结构的一致性。这种"块级 CRDT"设计是 Anytype any-sync 协议的核心思想之一，Aurora Note 在此基础上做了简化和优化，使其更适合单用户多设备场景（而非多人协作场景）。

5.4 同步架构：混合模式设计

Aurora Note 的同步层支持三种模式，用户可根据需求选择或混合使用：

同步模式	原理	适用场景	优劣势
自建服务器（C/S 模式）	用户部署 Aurora Sync Server（Rust 实现），客户端通过 WebSocket 长连接推送变更。服务器仅存储加密后的 CRDT 更新，不可解密内容。	日常使用，多设备稳定同步	优势: 稳定可靠、支持离线推送 劣势: 需要维护服务器
P2P 直连（P2P 模式）	设备通过 mDNS（局域网）或 STUN/TURN（互联网）发现彼此，直接建立加密通道同步 CRDT 更新。使用 libp2p 实现。	同一局域网内大文件同步、无服务器环境	优势: 零服务器成本、低延迟 劣势: 需要设备在线、NAT 穿透可能失败
第三方云存储（桥接模式）	将 CRDT 更新序列化为文件，通过 WebDAV/S3/Dropbox 等第三方存储同步。客户端定期轮询拉取更新。	无自建服务器能力的用户、临时使用	优势: 零运维成本 劣势: 延迟高、有轮询间隔

表 5-3 三种同步模式对比

三种模式可以混合使用：日常通过自建服务器同步，在旅行时（无服务器可达）切换到 P2P 直连，在共享笔记给非用户时通过桥接模式。同步层的设计使得切换模式对上层完全透明——核心逻辑层只关心“有新的 CRDT 更新需要合并”，不关心更新来自哪个通道。

5.5 端到端加密方案

Aurora Note 的 E2EE 方案参考 Notesnook 的设计，采用三层密钥体系：

密钥层级	用途	生成与存储
主密钥 (Master Key)	由用户密码经 Argon2id 密钥派生函数生成，用于加密密钥链。不离开设备。	$\text{Argon2id}(\text{password}, \text{salt}) \rightarrow 256\text{-bit key}$ 存储在系统 Keychain（macOS/iOS）/Keystore（Android）/Credential Manager（Windows）
笔记密钥 (Note Key)	每条笔记有独立的 XChaCha20-Poly1305 对称密钥，用于加密笔记内容。	CSPRNG 随机生成，用主密钥加密后存储在同步服务器。设备间通过密钥分享协议同步。
设备密钥 (Device Key)	每个设备有独立的 X25519 密钥对，用于设备间安全通信和密钥分享。	首次启动时生成，私钥不离开设备，公钥注册到同步服务器。

表 5-4 三层密钥体系

设备更换/丢失时，用户可以通过"恢复密钥"（Recovery Key，12 个助记词）在新设备上恢复访问权限。恢复密钥是对主密钥的加密备份，存储在同步服务器上。这一设计确保了即使所有设备丢失，用户数据也不会永久不可访问，同时保证了服务器端无法解密任何用户数据。

5.6 插件系统：WASM 沙箱架构

Aurora Note 的插件系统基于 WASM 沙箱，使用 wasmtime 运行时。插件可以用任何能编译为 WASM 的语言编写（推荐 Rust/AssemblyScript/Go），在独立内存空间中运行，通过能力声明（Capability-based Security）控制权限。

能力名称	权限说明	典型用途
note:read	读取指定笔记内容	AI 摘要插件读取笔记内容
note:write	修改指定笔记内容	AI 补全插件写入建议内容
fs:read	读取指定目录文件	图片插入插件读取本地图片
fs:write	写入指定目录文件	导出插件写入 PDF/Markdown 文件
net:fetch	发起 HTTP 请求（指定域名白名单）	网页剪藏插件抓取网页内容
ai:inference	调用 AI 推理接口	翻译/改写/标签推荐插件
ui:panel	注册 UI 面板/命令	侧边栏插件/命令面板插件

表 5-5 插件能力声明列表

插件在安装时必须声明所需能力，用户在安装前可以审查能力列表。运行时 wasmtime 强制执行能力限制——未声明的操作会被运行时拒绝。这种设计既保证了安全性（恶意插件无法越权访问），又保证了功能性（合法插件可以通过声明获取所需能力）。插件 API 使用 Protocol Buffers 定义接口，保证跨语言兼容性和版本演进能力。

5.7 AI 集成架构

Aurora Note 在架构层面预留 AI 集成接口，支持本地推理和云端推理的混合模式：

AI 能力	本地推理	云端推理
语义搜索	使用 all-MiniLM-L6-v2 等小型嵌入模型（ONNX Runtime），在本地构建向量索引（SQLite-VSS）	使用 OpenAI text-embedding-3-large 等高质量嵌入模型（可选）
问答辅助	使用 Phi-3-mini/Qwen2-1.5B 等量化小模型（llamacpp），在本地进行 RAG 问答	使用 GPT-4o/Claude 3.5 等大模型（用户配置 API Key）
内容补全	本地小模型提供基础补全建议	云端大模型提供高质量补全
智能标签	本地分类模型自动推荐标签	云端模型提供更准确的分类

表 5-6 AI 能力矩阵：本地 vs 云端

AI 推理的隐私保护机制：本地推理完全在设备上完成，数据不出端；云端推理时，用户可以配置"隐私过滤器"，自动脱敏（将人名/地名/敏感词替换为占位符）后再发送到云端 API。向量索引存储在本地 SQLite-VSS 中，不随同步传播（各设备独立构建索引），避免向量数据泄露。

第六章 技术选型论证与二次开发路线

本章详细论证 Aurora Note 各层技术选型的理由，并提出在优秀开源软件基础上进行二次开发的具体路线。核心思路是：不重新发明轮子，在已验证的开源项目基础上构建，将工程精力集中在差异化能力的开发上。

6.1 为什么选择 Flutter（移动端）+ Tauri（桌面端）的混合方案

在第三章的分析中，Flutter+Rust 和 Tauri+Web 都获得了 28/35 的并列最高分。Aurora Note 选择两者的混合方案，而非单一技术栈，理由如下：

移动端选 Flutter 的理由：Flutter 在移动端的成熟度远超 Tauri 2.0 移动端。经过大量生产应用验证，Flutter 的 iOS/Android 渲染一致性、手势处理、列表滚动性能、相机/传感器集成等方面都已非常成熟。笔记应用的移动端场景（通勤时记录、会议笔记、拍照采集）对 UI 流畅度要求高，Flutter 是当前最优选择。此外，AppFlowy 的 Flutter 编辑器可以直接复用，节省大量开发工作。

桌面端选 Tauri 的理由：桌面端笔记应用的核心是编辑器，而 Web 生态拥有最成熟的编辑器框架（ProseMirror/Lexical 等）。Tauri 允许使用 Web 技术构建前端（直接复用这些编辑器框架），同时使用 Rust 后端保证性能和安全性。相比 Electron，Tauri 的包体积小一个数量级（10MB vs 150MB），内存占用低 3-5 倍。对于笔记应用这种“常驻后台”的场景，资源占用的差异对用户体验影响显著。Tauri 2.0 的权限系统也比 Electron 更安全，适合实现插件沙箱。

核心层统一的理由：Flutter 端和 Tauri 端共享同一套 Rust 核心代码（L1-L3 层），通过不同的 FFI 桥接（flutter_rust_bridge vs Tauri Commands）暴露给上层 UI。这意味着数据模型、编辑器逻辑、同步引擎、加密模块全部只实现一次，两端行为完全一致。这是整个架构中最重要设计决策——它保证了“一套核心 + 两套 UI”的可行性，避免了 Joplin 式的“两套代码两套逻辑”问题。

6.2 为什么选择 Yjs/yrs 作为 CRDT 引擎

CRDT 引擎的选择主要有三个候选：Yjs/yrs（JavaScript 原版 + Rust 移植）、Automerge（JavaScript + Rust 核心）、自研 CRDT。Aurora Note 选择 Yjs/yrs，理由如下：

生态成熟度：Yjs 是目前最流行的 CRDT 库，在 GitHub 上有超过 17K Star，被 Notion/Figma/Linear 等产品的协作功能使用或参考。yrs 是 Yjs 的 Rust 实现，由 Yjs 作者亲自参与设计，API 与 Yjs 高度兼容。这意味着可以在 Rust 核心层使用 yrs，在需要 Web 端协作时使用 Yjs，两者可以互操作。

性能优势：Yjs 的二进制编码格式（Y-CRDT）在存储效率和合并速度上优于 Automerge。在处理大文档（10万+ 操作历史）时，yrs 的合并速度比 Automerge 快 2-3 倍，内存占用低 40%。对于笔记应用的大文档场景（如长篇笔记、数据库视图），这一性能差异显著。

不选 Automerge 的原因：Automerge 的 API 设计更优雅（不可变数据结构），但性能不如 Yjs，且 Rust 核心（automerge-rs）的成熟度不如 yrs。自研 CRDT 的方案被排除——CRDT 的数学正确性证明和工程实现都极其复杂，不值得自研。

6.3 二次开发路线：基于 AppFlowy 的 Fork 策略

Aurora Note 推荐在 AppFlowy 基础上进行二次开发，而非从零构建。以下是对三种路线的对比：

路线	工作量估计	优势	风险
路线 A: Fork AppFlowy	约 60% 工作量（节省约 40%）	复用成熟的 Flutter+Rust 架构、块级编辑器、数据层。社区活跃，可持续 merge 上游更新。	需要理解 AppFlowy 代码结构（约 15 万行）。AGPL-3.0 许可证要求开源修改（本报告不考虑商用因素）。
路线 B: Fork Anytype	约 70% 工作量	复用最先进的 P2P+CRDT 架构和 any-sync 协议。	Anytype 架构复杂度高，学习曲线陡峭。any-sync 协议的自建部署文档不完善。React 前端与目标技术栈（Flutter/Tauri）不一致，需要重写 UI。
路线 C: 从零构建	100% 工作量（约 18-24 个月）	完全自主可控，架构最优化。	研发周期长、风险高。块级编辑器和 CRDT 同步引擎的自研工作量巨大。

表 6-1 三种二次开发路线对比

推荐路线 A（Fork AppFlowy）的具体实施策略：

- **保留：**AppFlowy 的 Flutter UI 框架、Rust 数据层（flowy-core/flowy-database）、块级编辑器内核（flowy-editor）、SQLite 存储引擎。这些是 AppFlowy 最有价值的底层资产，经过大规模验证。
- **替换-同步层：**移除 AppFlowy 的 Supabase/WASM 同步后端，替换为基于 yrs CRDT 的自研同步引擎，支持自建服务器和 P2P 双模式。这是 Aurora Note 最核心的差异化能力。
- **替换-桌面端：**将桌面端从 Flutter Desktop 切换为 Tauri 2.0 + React。Flutter Desktop 虽然已稳定，但在桌面端的文本编辑体验（快捷键、拖拽、多窗口）不如 Web 技术灵活。Tauri 端通过 Rust FFI 直接调用同一套核心层，无需重复实现。

- **新增-E2EE**：在 Rust 核心层新增 crypto 模块，参考 Notesnook 的三层密钥体系实现端到端加密。这是 AppFlowy 目前缺失但用户日益重视的能力。
- **新增-WASM 插件**：在 Rust 核心层集成 wasmtime 运行时，实现基于能力声明的插件沙箱。AppFlowy 目前的插件系统尚不成熟，这是 Aurora Note 的差异化优势。
- **新增-AI 集成**：预留 AI 推理接口，首期实现本地语义搜索（all-MiniLM-L6-v2 + SQLite-VSS）和云端问答（OpenAI API），后续逐步扩展本地推理能力。
- **持续 merge 上游**：保持与 AppFlowy 主仓库的同步，定期 merge 上游更新（尤其是编辑器和数据层的 bug fix 和性能优化），避免 fork 分叉过大导致无法合并。

6.4 技术栈选型完整清单

层级	技术选型	版本	选型理由
移动端 UI	Flutter	3.x stable	移动端成熟度最高，一套代码覆盖 iOS/Android，AppFlowy 编辑器可直接复用
桌面端 UI	Tauri 2.0 + React	Tauri 2.0+	轻量（10MB vs Electron 150MB），Rust 后端安全，Web 生态复用 ProseMirror 等编辑器框架
核心层	Rust	stable	性能接近 C++，内存安全，无 GC 暂停，yrs/wasmtime/argon2 等关键库均为 Rust 原生
FFI 桥接	flutter_rust_bridge + Tauri Commands	2.x	自动生成 Dart-Rust 绑定，类型安全；Tauri Commands 原生支持 Rust 后端
本地存储	SQLite	3.45+	嵌入式数据库标杆，Rust 绑定（rusqlite）成熟，支持 FTS5 全文搜索扩展
向量索引	SQLite-VSS / LanceDB	latest	嵌入式向量数据库，支持本地语义搜索，与 SQLite 集成
CRDT 引擎	yrs (Yjs Rust)	0.x → 1.0	Yjs 生态最成熟的 Rust 实现，性能优于 Automerge，支持复杂文档模型
同步协议	WebSocket + libp2p	—	WebSocket 用于 C/S 同步（稳定），libp2p 用于 P2P 直连（灵活）
加密	XChaCha20-Poly1305 + X25519 + Argon2id	—	现代加密标准，性能优于 AES-GCM，libsodium Rust 绑定成熟
插件运行时	wasmtime	20.x+	Bytecode Alliance 的 WASM 运行时，支持能力声明安全模型，性能接近原生

层级	技术选型	版本	选型理由
AI 本地推理	ONNX Runtime + llama.cpp	—	ONNX Runtime 支持嵌入模型推理，llama.cpp 支持量化 LLM 本地运行
AI 云端推理	OpenAI API / Anthropic API	—	用户自行配置 API Key，支持多家供应商
桌面编辑器	ProseMirror / Lexical	—	Web 生态最成熟的块级编辑器框架，Tauri 前端直接使用
移动端编辑器	AppFlowy Editor (二次开发)	—	Flutter 生态唯一的成熟块级编辑器，在 AppFlowy 中经过大规模验证

表 6-2 Aurora Note 技术栈选型完整清单

第七章 实施路线图与团队配置

本章将 Aurora Note 的开发分为四个阶段，每个阶段有明确的目标、可交付物和团队配置建议。整体周期约 12-15 个月（基于 5-7 人核心团队），如果采用 Fork AppFlowy 路线可缩短至 9-12 个月。

7.1 四阶段实施路线

阶段	时间	目标	可交付物	核心团队
Phase 1 基础架构	M1-M3	搭建分层架构骨架，跑通"Flutter+Rust+Tauri"三端编译	项目脚手架、FFI 桥接层、SQLite 数据层、基础 CRUD API、CI/CD 流水线	2 Rust + 1 Flutter + 1 Tauri/前端
Phase 2 核心功能	M4-M7	实现编辑器、同步、加密三大核心能力	块级编辑器（移动端+桌面端）、CRDT 同步引擎、E2EE 加密模块、自建同步服务器 MVP	3 Rust + 2 Flutter + 1 Tauri/前端 + 1 后端
Phase 3 生态能力	M8-M11	插件系统、AI 集成、高级功能	WASM 插件沙箱、插件 API + SDK、本地语义搜索、云端 AI 问答、数据库视图、双向链接	3 Rust + 2 Flutter + 1 Tauri/前端 + 1 AI + 1 测试
Phase 4 打磨发布	M12-M15	性能优化、Bug 修复、应用商店发布	性能基准达标（搜索<50ms/编辑>55fps/启动<2s）、各平台应用商店上架、文档与开发者指南	全员 + 1 设计 + 1 DevRel

表 7-1 四阶段实施路线图

7.2 团队配置建议

Aurora Note 的架构对团队技能有较高要求——需要同时具备 Rust、Flutter 和 Web/React 三种技能的工程师。以下是推荐的 7 人核心团队配置：

角色	人数	核心技能要求	主要职责
Rust 核心工程师	3	Rust（高级）、SQLite、CRDT/密码学（各 1 人专精）	数据层、同步引擎、加密模块、WASM 插件运行时的开发和维护
Flutter 移动端工程师	2	Flutter（高级）、Dart、移动端原生集成	移动端 UI、编辑器适配、flutter_rust_bridge 集成、移动端性能优化

角色	人数	核心技能要求	主要职责
Tauri/前端工程师	1	React（高级）、TypeScript、Tauri、ProseMirror/Lexical	桌面端 UI、Web 编辑器集成、Tauri Commands 桥接
AI/后端工程师	1	Python/Rust、LLM 集成、向量数据库、WebSocket	AI 推理接口、同步服务器、向量索引、RAG 管线

表 7-2 推荐团队配置（7 人核心团队）

Phase 4（打磨发布阶段）可以额外增加 1 名设计师（负责 UI/UX 打磨和图标设计）和 1 名 DevRel（负责开发者文档和社区运营），使总团队规模达到 9 人。测试工作由全员参与（代码审查 + 集成测试），Phase 3 开始可增加 1 名专职 QA 工程师。

第八章 风险评估与缓解策略

任何技术架构在实施过程中都面临风险。本章从技术风险、工程风险和生态风险三个维度进行评估，并提出相应的缓解策略。风险评估的目的是提前识别潜在问题，制定应对预案，而非回避风险。

8.1 技术风险

风险	概率	影响	缓解策略
Tauri 2.0 移动端不成熟，桌面端 WebView 碎片化	中	高	桌面端优先支持 Win/Mac（WebView2/WebKit 稳定），Linux 延后。移动端完全使用 Flutter，不依赖 Tauri 移动端。
yrs CRDT 在极端场景下的正确性问题	低	高	定期进行模糊测试（fuzzing）验证 CRDT 合并正确性。保留 LWW 回退方案——当 CRDT 合并出现异常时，降级为版本号比较。
Flutter 桌面端编辑器体验不足	中	中	桌面端切换到 Tauri+ProseMirror，移动端保留 Flutter 编辑器。两端共享核心层的数据模型，UI 各自优化。
WASM 插件性能不如预期	低	中	核心插件（如 AI 辅助）用 Rust 原生实现，不依赖 WASM。WASM 仅用于第三方插件。性能基准：WASM 插件操作延迟 <5ms。
本地 AI 推理资源消耗过大	中	中	本地推理默认关闭，用户可选项启用。仅使用量化小模型（<1GB），在低端设备上自动降级为仅云端推理。

表 8-1 技术风险评估

8.2 工程风险

风险	概率	影响	缓解策略
Fork AppFlowy 后上游分叉过大，无法 merge 更新	中	高	对 AppFlowy 的修改尽量通过插件/配置方式实现，减少核心代码改动。定期（每月）同步上游更新。维护补丁清单和变更日志。
Rust 工程师招聘困难	高	高	核心层代码模块化，接口定义清晰，降低上手难度。为 Flutter/前端工程师提供 Rust 培训。考虑远程招聘 Rust 工程师。

风险	概率	影响	缓解策略
跨端 FFI 桥接调试困难	中	中	建立完善的 FFI 集成测试套件。使用 flutter_rust_bridge 的自动绑定生成，减少手工桥接代码。日志系统跨端统一。
开发周期超预期	中	中	Phase 1 设为"最小可编译"目标，尽早验证三端架构可行性。每 Phase 结束进行 retrospective，调整后续计划。

表 8-2 工程风险评估

8.3 生态风险

风险	概率	影响	缓解策略
AppFlowy 社区改变许可证（如从 AGPL 改为更严格许可）	低	高	定期备份 AppFlowy 仓库。在 Fork 时记录所有依赖项的许可证。核心层保持独立，不深度耦合 AppFlowy 特定 API。
yrs 库停止维护或被废弃	低	高	CRDT 层抽象为 trait，可替换为 Automerge 或自研实现。yrs 代码开源，必要时可自行维护。
wasmtime 运行时方向变化	低	中	WASM 插件 API 基于 WASI 标准，不绑定特定运行时。可替换为 wasmer 或 WasmEdge。
AI 模型/API 供应链变化（如 OpenAI 限制访问）	中	低	AI 接口抽象为 provider trait，支持多家供应商。本地推理能力作为 fallback。

表 8-3 生态风险评估

8.4 总结与建议

综合以上评估，Aurora Note 架构的主要风险集中在三个方面：（1）Tauri 2.0 移动端成熟度——通过 Flutter 独立处理移动端来规避；（2）Rust 人才招聘——通过远程招聘和团队培训来缓解；（3）AppFlowy Fork 的上游同步——通过最小化核心代码改动和定期同步来管理。这些风险都是可管理的，不构成架构可行性的根本障碍。

建议在 Phase 1 结束时进行"架构验证评审"——评估三端编译是否跑通、FFI 桥接是否稳定、核心数据层是否满足性能要求。如果 Phase 1 验证不通过，需要及时调整技术方案（如桌面端从 Tauri 切换到 Flutter Desktop），而不是继续推进到 Phase 2。这种"尽早失败"的策略可以最大限度降低后期返工的成本。

本报告覆盖的 10 款产品、5 大技术栈和 8 项演进趋势的分析，为 Aurora Note 的架构设计提供了充分的行业对标基础。Aurora Note 的 Flutter（移动端）+ Tauri（桌面端）+ Rust（核心层）混合架构，结合 CRDT 同步、E2EE 加密、WASM 插件和 AI 集成，在长期可演进性、可扩展性、灵活性和性能之间取得了最佳平衡。通过 Fork AppFlowy 的二次开发路线，可以将开发周期从 18-24 个月缩短至 9-12 个月，同时保持架构的独立性和差异化能力。